

Capítulo 1: Bases de Datos y Usuarios de Bases de Datos

1.1 Introducción

En el mundo actual, es prácticamente imposible pasar un día sin interactuar con alguna **base de datos**. Piensen, por ejemplo, cuando usan su aplicación bancaria, hacen una reserva de hotel, buscan un libro en el catálogo de una biblioteca o incluso cuando compran algo en el supermercado; en todos esos casos, sus acciones están interactuando con una **base de datos**.

Tradicionalmente, las **bases de datos** almacenaban información textual o numérica. Sin embargo, los avances tecnológicos nos han llevado a nuevas y emocionantes aplicaciones. Hoy en día, las redes sociales, con sus inmensos volúmenes de publicaciones, imágenes y videos, requieren sistemas de **bases de datos** que gestionen datos no tradicionales. Han surgido nuevos tipos, a menudo llamados **sistemas de almacenamiento de big data** o **sistemas NOSQL**, para manejar estas necesidades masivas. Empresas como Google y Amazon los utilizan para sus motores de búsqueda y servicios de almacenamiento en la nube, permitiéndoles gestionar todo tipo de datos.

Pero no solo eso. Las **bases de datos** también son fundamentales en otras áreas:

- Almacenan imágenes, clips de audio y video en **bases de datos multimedia**.
- Los **sistemas de información geográfica (SIG)** manejan mapas, datos meteorológicos e imágenes satelitales.
- Los **almacenes de datos** (*data warehouses*) y los **sistemas de procesamiento analítico en línea (OLAP)** se usan en empresas para extraer y analizar información de negocio para la toma de decisiones.
- La **tecnología de bases de datos en tiempo real y activas** se aplica para controlar procesos industriales.
- Las técnicas de búsqueda en **bases de datos** mejoran la recuperación de información en la *World Wide Web*.

Para entender todo esto, empezaremos desde lo básico. Una **base de datos**, en su definición más general, es una colección de datos relacionados. Por **datos**, entendemos hechos conocidos que pueden ser registrados y que tienen un significado implícito. Por ejemplo, los nombres, números de teléfono y direcciones de sus contactos en el móvil constituyen una **base de datos**.

Aunque esta definición es amplia, el término **base de datos** en su uso común tiene propiedades más específicas:

- Una **base de datos** representa algún aspecto del mundo real, al que llamamos **mini-mundo** o **universo del discurso (UoD)**. Los cambios en este **mini-mundo** deben reflejarse en la **base de datos**.
- Es una colección de datos lógicamente coherente y con un significado inherente. No cualquier conjunto aleatorio de datos es una **base de datos**.

- Está diseñada, construida y poblada con datos para un propósito específico, con un grupo de usuarios y aplicaciones predefinidas en mente.

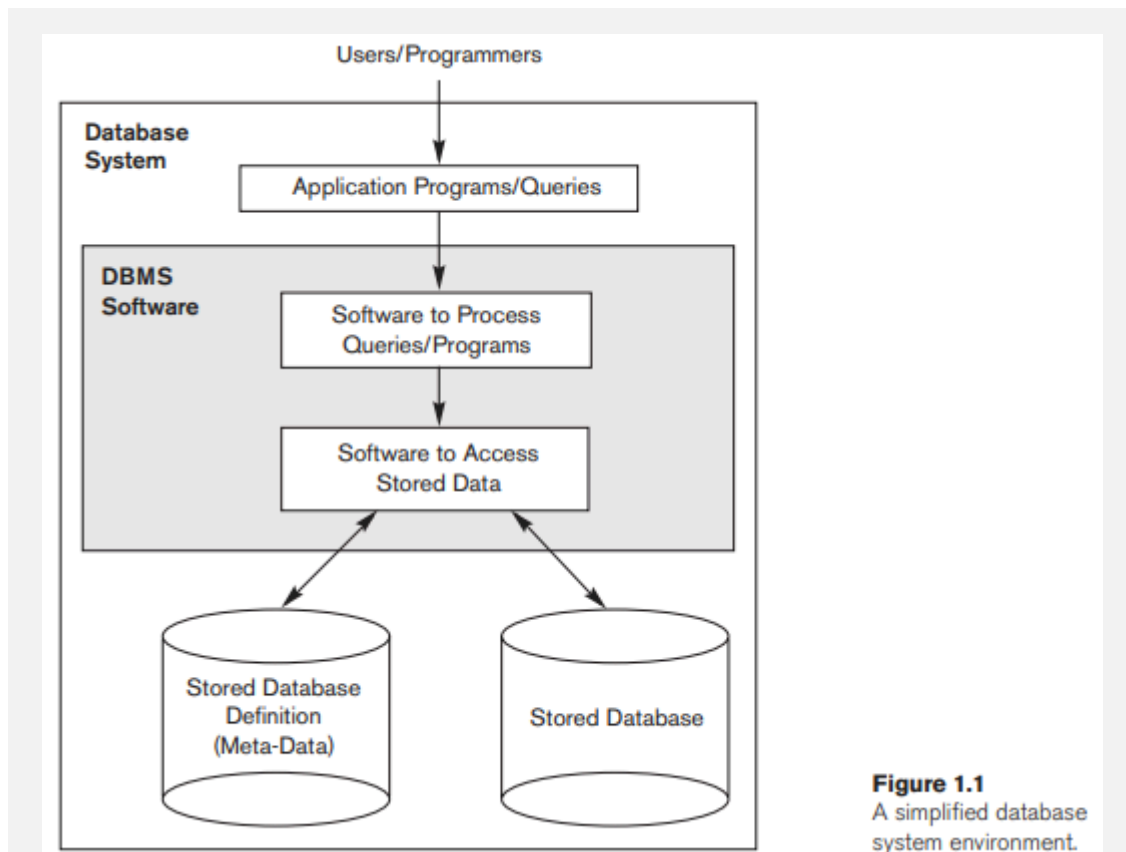


Figure 1.1
A simplified database
system environment.

Observen la *Figura 1.1*, que ilustra un entorno simplificado de sistema de base de datos. Allí pueden ver cómo los usuarios y programadores interactúan con el software **DBMS** a través de programas de aplicación y consultas. El **DBMS** a su vez, gestiona el acceso a la **base de datos** almacenada y su definición (**meta-datos**). Este es el esquema básico que nos guiará a lo largo del curso.

En esencia, una **base de datos** tiene una fuente de la que derivan los datos, interactúa con eventos del mundo real y tiene una audiencia interesada en su contenido. Para que sea precisa y confiable, debe reflejar los cambios del **mini-mundo** tan pronto como sea posible. Las **bases de datos** pueden ser de cualquier tamaño y complejidad, desde unas pocas centenas de registros, como una lista de direcciones, hasta millones de entradas, como el catálogo de una gran biblioteca o los sistemas de una compañía como Amazon.com, que gestiona datos para más de 60 millones de usuarios activos y millones de productos, ocupando terabytes de almacenamiento.

Una **base de datos** puede ser mantenida manualmente o computarizada. En este curso, nos enfocaremos en las **bases de datos computarizadas**. Aquí es donde entra en juego el **Sistema de Gestión de Bases de Datos (DBMS)**.

Un **DBMS** es un software que permite a los usuarios crear y mantener una **base de datos**. Facilita los procesos de:

- **Definir una base de datos:** Especificar tipos de datos, estructuras y restricciones. Esta definición, llamada **meta-datos**, se almacena en el **catálogo de la base de datos** o **diccionario de datos**.
- **Construir la base de datos:** Almacenar los datos en un medio controlado por el **DBMS**.
- **Manipular una base de datos:** Consultar (recuperar datos), actualizar (reflejar cambios) y generar informes.
- **Compartir una base de datos:** Permitir que múltiples usuarios y programas accedan simultáneamente.

Los programas de aplicación interactúan con la **base de datos** enviando **consultas** (*queries*) o solicitudes de datos al **DBMS**. Una **consulta** suele recuperar datos, mientras que una **transacción** puede leer y escribir datos.

Además, el **DBMS** tiene funciones cruciales como proteger la **base de datos** contra fallos de hardware/software y accesos no autorizados, y mantenerla a lo largo del tiempo, permitiendo su evolución conforme cambian los requisitos.

Aunque es posible crear programas personalizados para gestionar una **base de datos**, el **DBMS** es un sistema de software de propósito general que centraliza y simplifica estas tareas. En conjunto, la **base de datos** y el software **DBMS** forman un **sistema de base de datos**.

1.2 Un Ejemplo

Para concretar estos conceptos, consideremos un ejemplo familiar: una **base de datos** de **UNIVERSIDAD**. Esta **base de datos** gestiona información sobre estudiantes, cursos y calificaciones.

En la *Figura 1.2*, pueden ver la estructura de esta **base de datos** con algunos datos de ejemplo. Se organiza en cinco archivos, cada uno para un tipo de registro:

- **STUDENT (ESTUDIANTE):** Guarda datos de cada estudiante (Nombre, Número_de_estudiante, Clase, Carrera).
- **COURSE (CURSO):** Almacena datos de cada curso (Nombre_de_curso, Número_de_curso, Horas_crédito, Departamento).
- **SECTION (SECCIÓN):** Contiene información sobre cada sección de un curso (Identificador_de_sección, Número_de_curso, Semestre, Año, Instructor).
- **GRADE_REPORT (INFORME_DE_CALIFICACIONES):** Registra las calificaciones de los estudiantes en las secciones completadas (Número_de_estudiante, Identificador_de_sección, Calificación).
- **PREREQUISITE (PRERREQUISITO):** Guarda los requisitos previos de cada curso (Número_de_curso, Número_de_prerrequisito).

Para definir esta **base de datos**, debemos especificar la estructura de los registros de cada archivo, incluyendo los tipos de elementos de datos y sus tipos de datos (por ejemplo, Nombre del ESTUDIANTE es una cadena de caracteres, Número_de_estudiante del ESTUDIANTE es un entero).

Observen cómo los registros en diferentes archivos están relacionados. Por ejemplo, el registro de 'Smith' en el archivo STUDENT se relaciona con dos registros en GRADE_REPORT que especifican sus calificaciones.

La manipulación de la **base de datos** implica **consultas** y **actualizaciones**. Algunos ejemplos de **consultas** serían:

- "Recuperar el expediente académico (lista de cursos y calificaciones) de 'Smith'."
- "Listar los nombres de los estudiantes que tomaron la sección del curso 'Base de Datos' ofrecida en otoño de 2008 y sus calificaciones."
- "Listar los prerrequisitos del curso 'Base de Datos'."

Y ejemplos de **actualizaciones**:

- "Cambiar la clase de 'Smith' a *sophomore*."
- "Crear una nueva sección para el curso 'Base de Datos' para este semestre."
- "Ingresar una calificación de 'A' para 'Smith' en la sección de 'Base de Datos' del semestre pasado."

Estas operaciones informales deben ser especificadas con precisión en el lenguaje de consulta del **DBMS** para ser procesadas.

Este ejemplo nos permite entender que una **base de datos** es parte de un sistema de información más amplio. El diseño de una **base de datos** pasa por varias fases:

- **Especificación y análisis de requisitos:** Documentar lo que el usuario necesita.
- **Diseño conceptual:** Transformar los requisitos en un modelo de alto nivel, a menudo usando el **modelo Entidad-Relación (ER)** (que estudiaremos en el *Capítulo 3*).
- **Diseño lógico:** Traducir el diseño conceptual a un modelo de datos implementable en un **DBMS** comercial (como los **DBMS** relacionales en los *Capítulos 5 a 9*).
- **Diseño físico:** Especificar cómo se almacenarán y accederán los datos.

1.3 Características del Enfoque de Bases de Datos

El **enfoque de bases de datos** se distingue notablemente del **procesamiento de archivos tradicional**. En este último, cada usuario o departamento (como la oficina de registro y la de contabilidad) creaba y gestionaba sus propios archivos, lo que llevaba a:

- **Redundancia de datos:** La misma información se almacenaba múltiples veces.
- Desperdicio de espacio de almacenamiento.
- **Inconsistencia de datos:** Las actualizaciones no se aplicaban uniformemente en todos los archivos, llevando a información contradictoria.

En contraste, el **enfoque de base de datos** utiliza un único repositorio de datos definido una vez y accedido por múltiples usuarios. Sus características principales son:

- Naturaleza autodescriptiva del sistema de base de datos.
- Aislamiento entre programas y datos, y abstracción de datos.

- Soporte de múltiples vistas de los datos.
- Compartición de datos y procesamiento de transacciones multiusuario.

STUDENT

Name	Student_number	Class	Major
Smith	17	1	CS
Brown	8	2	CS

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	07	King
92	CS1310	Fall	07	Anderson
102	CS3320	Spring	08	Knuth
112	MATH2410	Fall	08	Chang
119	CS1310	Fall	08	Anderson
135	CS3380	Fall	08	Stone

GRADE_REPORT

Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

PREREQUISITE

Course_number	Prerequisite_number
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310

Figure 1.2
A database that stores
student and course
information.

Una vez implementada, la **base de datos** se puebla con datos y se mantiene continuamente para reflejar el estado del **mini-mundo**.

Vamos a profundizar en cada una de ellas.

1.3.1 Naturaleza Autodescriptiva de un Sistema de Base de Datos

Una característica fundamental es que el **sistema de base de datos** contiene no solo los datos, sino también una definición completa de su estructura y restricciones. Esta definición se guarda en el **catálogo del DBMS**, también conocido como **meta-datos**. El **catálogo** incluye información como la estructura de cada archivo, el tipo y formato de almacenamiento de cada elemento de datos, y diversas restricciones.

El **DBMS** utiliza estos **meta-datos** para entender la estructura de cualquier **base de datos** que gestione, lo que le permite funcionar con diversas aplicaciones (universidad, banca, empresa) siempre que la definición esté en el **catálogo**. En el **procesamiento de archivos tradicional**, la definición de los datos estaba integrada en los programas de aplicación, lo que los ataba a una estructura específica.

RELATIONS

Relation_name	No_of_columns
STUDENT	4
COURSE	4
SECTION	5
GRADE_REPORT	3
PREREQUISITE	2

COLUMNS

Column_name	Data_type	Belongs_to_relation
Name	Character (30)	STUDENT
Student_number	Character (4)	STUDENT
Class	Integer (1)	STUDENT
Major	Major_type	STUDENT
Course_name	Character (10)	COURSE
Course_number	XXXXNNNN	COURSE
....		
....		
....		
Prerequisite_number	XXXXNNNN	PREREQUISITE

Note: Major_type is defined as an enumerated type with all known majors.
XXXXNNNN is used to define a type with four alphabetic characters followed by four numeric digits.

Figure 1.3

An example of a database catalog for the database in Figure 1.2.

Volviendo a nuestro ejemplo de la UNIVERSIDAD, el **catálogo del DBMS** almacenaría las definiciones de todos los archivos que vimos. Por ejemplo, al acceder al Nombre de un registro STUDENT, el **DBMS** consultaría el **catálogo** para saber la estructura del archivo STUDENT y la posición y tamaño del campo Nombre. La *Figura 1.3* nos muestra un ejemplo de cómo se vería una parte de este **catálogo**, indicando los nombres de las relaciones (tablas), el número de columnas y detalles de cada columna como su nombre, tipo de dato y a qué relación pertenece.

1.3.2 Aislamiento entre Programas y Datos, y Abstracción de Datos

En el **procesamiento de archivos**, un cambio en la estructura de un archivo a menudo requiere modificar todos los programas que lo usan. En cambio, en un entorno **DBMS**, esto rara vez es necesario. Esto se conoce como **independencia programa-datos**. Por ejemplo, si decidiéramos añadir la fecha de nacimiento a cada registro de STUDENT, solo necesitaríamos cambiar la descripción del registro STUDENT en el **catálogo**; los programas existentes no necesitarían ser modificados.

Además, en sistemas como las **bases de datos orientadas a objetos** o **bases de datos objeto-relacionales**, ustedes pueden definir operaciones (funciones o métodos) sobre los datos. La interfaz (nombre y tipos de argumentos) de una operación se define por separado de su implementación. Esto significa que pueden cambiar cómo se implementa una operación sin afectar los programas de usuario que la invocan, un concepto llamado **independencia programa-operación**.

La propiedad que permite tanto la **independencia programa-datos** como la **independencia programa-operación** es la **abstracción de datos**. Un **DBMS** les presenta una representación conceptual de los datos que oculta los detalles de almacenamiento e implementación, utilizando **modelos de datos** con conceptos lógicos como objetos, propiedades y relaciones, más fáciles de entender que los conceptos de almacenamiento en la computadora.

Data Item Name	Starting Position in Record	Length in Characters (bytes)
Name	1	30
Student_number	31	4
Class	35	1
Major	36	4

Figure 1.4
Internal storage format
for a STUDENT record,
based on the database
catalog in Figure 1.3.

Por ejemplo, la *Figura 1.4* muestra cómo se vería la representación interna de un registro STUDENT a bajo nivel, especificando la posición y longitud de cada campo en bytes. Sin embargo, un usuario típico solo se preocupa por acceder al "Nombre del ESTUDIANTE", no por dónde se almacena físicamente. El **DBMS** se encarga de estos detalles de almacenamiento, que discute más a fondo en los *Capítulos 16 y 17*.

1.3.3 Soporte de Múltiples Vistas de los Datos

Una **base de datos** suele tener muchos tipos de usuarios, y cada uno puede necesitar una perspectiva (o **vista**) diferente de la misma información. Una **vista** puede ser un subconjunto de la **base de datos** o contener **datos virtuales** derivados de los archivos de la **base de datos** pero no almacenados explícitamente.

1.3.4 Compartición de Datos y Procesamiento de Transacciones Multiuser

Un **DBMS multiusuario** debe permitir que muchos usuarios accedan a la **base de datos** simultáneamente. Para esto, el **DBMS** incluye un software de **control de concurrencia** que asegura que las actualizaciones simultáneas se realicen de manera controlada y correcta. Por ejemplo, si varios agentes de reservas intentan asignar el mismo asiento en un vuelo, el **DBMS** debe garantizar que solo un agente acceda a ese asiento a la vez. Este tipo de aplicaciones se llaman **aplicaciones de procesamiento de transacciones en línea (OLTP)**.

TRANSCRIPT					
Student_name	Student_transcript				
	Course_number	Grade	Semester	Year	Section_id
Smith	CS1310	C	Fall	08	119
	MATH2410	B	Fall	08	112
Brown	MATH2410	A	Fall	07	85
	CS1310	A	Fall	07	92
	CS3320	B	Spring	08	102
	CS3380	A	Fall	08	135

(a)

COURSE_PREREQUISITES		
Course_name	Course_number	Prerequisites
Database	CS3380	CS3320
		MATH2410
Data Structures	CS3320	CS1310

(b)

Figure 1.5

Two views derived from the database in Figure 1.2. (a) The TRANSCRIPT view.
(b) The COURSE_PREREQUISITES view.

Un **DBMS multiusuario** debe permitir la definición de múltiples vistas. Por ejemplo, un usuario puede estar interesado solo en el expediente académico de los estudiantes, mientras que otro solo en los prerrequisitos de los cursos. La *Figura 1.5* ilustra esto: (a) muestra la vista TRANSCRIPT (expediente académico), y (b) la vista COURSE_PREREQUISITES (prerrequisitos del curso), ambas derivadas de la misma **base de datos** de la UNIVERSIDAD. Esto permite que cada usuario se enfoque solo en la información relevante para su tarea.

El concepto de **transacción** es central aquí. Una **transacción** es un programa o proceso en ejecución que incluye una o más operaciones de acceso a la **base de datos** (lectura o actualización). Cada **transacción** debe ejecutarse de forma lógicamente correcta, sin interferencia de otras transacciones. El **DBMS** debe asegurar dos propiedades clave:

- **Aislamiento:** Cada **transacción** parece ejecutarse de forma aislada de otras, incluso si cientos se ejecutan concurrentemente.
- **Atomicidad:** Todas las operaciones de una **transacción** se ejecutan, o ninguna lo hace.

Discutiremos las transacciones en detalle en la *Parte 9*. Estas características son clave para diferenciar un **DBMS** del software tradicional de **procesamiento de archivos**.

1.4 Actores en Escena

En una gran organización, muchas personas participan en el diseño, uso y mantenimiento de una **base de datos**. A estas personas las llamamos los **actores en escena**.

1.4.1 Administradores de Bases de Datos

En cualquier organización, es fundamental contar con un administrador que supervise los recursos compartidos. En el entorno de **bases de datos**, este rol lo desempeña el **Administrador de Bases de Datos (DBA)**. Las responsabilidades del **DBA** incluyen:

- Autorizar el acceso a la **base de datos**.
- Coordinar y monitorear su uso.
- Adquirir recursos de software y hardware necesarios.
- Es el responsable de problemas como violaciones de seguridad o bajo rendimiento del sistema.

En grandes organizaciones, el **DBA** suele tener un equipo que le asiste en estas funciones.

1.4.2 Diseñadores de Bases de Datos

Los **diseñadores de bases de datos** son quienes identifican qué datos se almacenarán y cómo se estructurarán. Esta labor se realiza antes de la implementación de la **base de datos**. Su función es comunicarse con todos los posibles usuarios para entender sus requisitos y crear un diseño que los satisfaga. A menudo, desarrollan vistas para cada grupo de usuarios y luego las integran para asegurar que el diseño final cubra todas las necesidades.

1.4.3 Usuarios Finales

Los **usuarios finales** son las personas cuya labor principal requiere acceder a la **base de datos** para consultar, actualizar y generar informes. Existen varias categorías:

- **Usuarios finales casuales:** Acceden ocasionalmente y necesitan información diferente cada vez. Usan interfaces de consulta sofisticadas y suelen ser gerentes o exploradores ocasionales.
- **Usuarios finales ingenuos o paramétricos:** Constituyen una gran parte de los usuarios. Su trabajo se centra en consultar y actualizar la **base de datos** usando tipos de consultas y actualizaciones estándar, llamadas **transacciones enlatadas**. Ejemplos incluyen:
 - Clientes y cajeros de banco verificando saldos.
 - Agentes de reservas de aerolíneas haciendo reservaciones.
 - Empleados que ingresan información de paquetes en empresas de envío.

- Usuarios de redes sociales que publican y leen elementos. Muchas de estas tareas ahora se realizan a través de aplicaciones móviles.
- **Usuarios finales sofisticados:** Ingenieros, científicos, analistas de negocio que dominan las facilidades del **DBMS** para implementar sus propias aplicaciones.
- **Usuarios autónomos:** Mantienen **bases de datos** personales usando paquetes de software listos para usar con interfaces basadas en menús o gráficos.

El **DBMS** ofrece diversas facilidades de acceso, desde las interfaces simples para usuarios ingenuos hasta las más complejas para usuarios sofisticados.

1.4.4 Analistas de Sistemas y Programadores de Aplicaciones (Ingenieros de Software)

Los **analistas de sistemas** definen los requisitos de los usuarios finales, especialmente los ingenuos y paramétricos, y desarrollan las especificaciones para las **transacciones enlatadas**. Los **programadores de aplicaciones** implementan estas especificaciones, las prueban, depuran, documentan y mantienen. Estos profesionales, a menudo llamados **desarrolladores de software** o **ingenieros de software**, deben conocer a fondo las capacidades del **DBMS**.

1.5 Trabajadores Tras Bambalinas

Además de quienes diseñan, usan y administran una **base de datos**, hay otros que participan en el diseño, desarrollo y operación del software **DBMS** y su entorno. Ellos no están directamente interesados en el contenido de la **base de datos** en sí, pero su trabajo es fundamental. Incluyen:

- **Diseñadores e implementadores de sistemas DBMS:** Diseñan e implementan los módulos e interfaces del **DBMS** como un paquete de software. Un **DBMS** es un sistema complejo con componentes para el **catálogo**, procesamiento de consultas, **control de concurrencia**, recuperación de datos, seguridad, etc. También deben interactuar con otros software del sistema, como el sistema operativo y los compiladores.
- **Desarrolladores de herramientas:** Diseñan e implementan herramientas de software que facilitan el modelado y diseño de **bases de datos**, el diseño de sistemas y la mejora del rendimiento. Pueden incluir paquetes para diseño de **bases de datos**, monitoreo de rendimiento, interfaces de lenguaje natural o gráficos, prototipado, simulación y generación de datos de prueba.
- **Personal de operación y mantenimiento (personal de administración de sistemas):** Son responsables de la ejecución y el mantenimiento del entorno de hardware y software del sistema de **base de datos**.

Aunque son esenciales para que el **sistema de base de datos** esté disponible, estos profesionales generalmente no usan el contenido de la **base de datos** para sus propios fines.

1.6 Ventajas de Usar el Enfoque DBMS

Ya hemos discutido algunas ventajas al comparar el enfoque **DBMS** con el procesamiento de archivos. Ahora, veamos otras capacidades que un buen **DBMS** debería poseer y cómo benefician a las organizaciones.

1.6.1 Control de la Redundancia

En el desarrollo de software tradicional, cada grupo de usuarios mantenía sus propios archivos, lo que generaba redundancia y los problemas que ya mencionamos:

- **Duplicación de esfuerzo:** Al ingresar datos en múltiples lugares.
- Desperdicio de espacio de almacenamiento.
- **Inconsistencia de datos:** Si las actualizaciones no se aplicaban de forma uniforme.

En el enfoque de base de datos, las vistas de los diferentes grupos de usuarios se integran durante el diseño de la base de datos. Idealmente, cada elemento lógico de datos (como el nombre o la fecha de nacimiento de un estudiante) se almacena en un solo lugar. Esto se conoce como **normalización de datos** y asegura la consistencia y ahorra espacio.

Figure 1.6

Redundant storage of Student_name and Course_name in GRADE_REPORT.
(a) Consistent data.
(b) Inconsistent record.

GRADE_REPORT

Student_number	Student_name	Section_identifier	Course_number	Grade
17	Smith	112	MATH2410	B
17	Smith	119	CS1310	C
8	Brown	85	MATH2410	A
8	Brown	92	CS1310	A
8	Brown	102	CS3320	B
8	Brown	135	CS3380	A

(a)

GRADE_REPORT

Student_number	Student_name	Section_identifier	Course_number	Grade
17	Brown	112	MATH2410	B

(b)

Sin embargo, a veces, es necesario permitir una **redundancia controlada** para mejorar el rendimiento de las consultas, un proceso llamado **desnormalización**. Por ejemplo, en el archivo GRADE_REPORT, podríamos almacenar redundancia de Student_name y Course_number para facilitar la recuperación. La *Figura 1.6* ilustra esto: (a) muestra datos consistentes con **redundancia controlada**, mientras que (b) muestra un registro inconsistente. En estos casos, el **DBMS** debe controlar la redundancia para evitar inconsistencias, realizando verificaciones automáticas.

1.6.2 Restricción del Acceso No Autorizado

Cuando una base de datos es compartida, no todos los usuarios tienen autorización para acceder a toda la información. Los datos confidenciales, como salarios, solo deben ser accesibles para personas autorizadas. Además, el tipo de operación (recuperación o actualización) también debe ser controlado.

Un **DBMS** debe proporcionar un **subsistema de seguridad y autorización**. El **DBA** lo utiliza para crear cuentas y especificar restricciones, que el **DBMS** luego aplica automáticamente. Esto incluye controlar

quién puede usar ciertas funcionalidades del software **DBMS** y qué transacciones predefinidas pueden ejecutar los usuarios paramétricos. Este tema se aborda en detalle en el *Capítulo 30*.

1.6.3 Provisión de Almacenamiento Persistente para Objetos de Programa

Las bases de datos pueden ofrecer **almacenamiento persistente** para objetos y estructuras de datos de programas. En lenguajes de programación, los valores de las variables u objetos se pierden al terminar el programa, a menos que se guarden explícitamente en archivos, lo que a menudo implica conversiones de formato.

Los **sistemas de bases de datos orientadas a objetos** son compatibles con lenguajes como C++ y Java. El **DBMS** realiza automáticamente las conversiones, permitiendo que un objeto complejo en C++ se almacene permanentemente y sea recuperado directamente por otro programa. A estos objetos se les llama **persistentes**.

Los sistemas de bases de datos tradicionales a menudo sufrían del problema de **desajuste de impedancia**, ya que las estructuras de datos del **DBMS** eran incompatibles con las de los lenguajes de programación. Los **DBMS orientados a objetos** buscan resolver esto ofreciendo compatibilidad.

1.6.4 Provisión de Estructuras de Almacenamiento y Técnicas de Búsqueda para un Procesamiento Eficiente de Consultas

Los sistemas de bases de datos deben ser eficientes para ejecutar consultas y actualizaciones. Como los datos se almacenan en disco, el **DBMS** utiliza estructuras de datos y técnicas de búsqueda especializadas para acelerar la recuperación de registros. Los archivos auxiliares llamados **índices** (basados en estructuras de árbol o *hash*) se usan para este fin.

Para procesar una consulta, los registros necesarios deben copiarse del disco a la memoria principal. Por ello, el **DBMS** a menudo tiene su propio módulo de almacenamiento en búfer o caché para gestionar las lecturas y escrituras en disco, lo cual es crucial para el rendimiento.

El módulo de **procesamiento y optimización de consultas** del **DBMS** elige un plan de ejecución eficiente para cada consulta, basándose en las estructuras de almacenamiento existentes. La elección de qué **índices** crear y mantener es parte del diseño físico de la base de datos y la sintonización, responsabilidad del equipo del **DBA**. Discutiremos esto en la *Parte 8* del texto.

1.6.5 Provisión de Copias de Seguridad y Recuperación

Un **DBMS** debe incluir mecanismos para recuperarse de fallos de hardware o software. El **subsistema de copias de seguridad y recuperación** del **DBMS** se encarga de esto. Por ejemplo, si el sistema falla en medio de una transacción compleja, el subsistema de recuperación asegura que la base de datos se restaure al estado previo al inicio de la transacción. Las copias de seguridad en disco son vitales en caso de fallos catastróficos. Esto lo exploraremos en el *Capítulo 22*.

1.6.6 Provisión de Múltiples Interfaces de Usuario

Dado que diversos tipos de usuarios con distintos niveles de conocimiento técnico interactúan con una base de datos, un **DBMS** debe ofrecer una variedad de interfaces:

- **Aplicaciones para dispositivos móviles:** Permiten a los usuarios móviles acceder a sus datos (banca, reservas, seguros).
- **Interfaces basadas en menús para clientes web o navegación:** Presentan listas de opciones para guiar al usuario en la formulación de una solicitud, eliminando la necesidad de memorizar comandos.
- **Interfaces basadas en formularios:** Muestran un formulario que los usuarios pueden rellenar para insertar o recuperar datos. Son comunes para transacciones enlatadas y usuarios ingenuos.
- **Interfaces gráficas de usuario (GUI):** Muestran el esquema de la base de datos en forma de diagrama, permitiendo al usuario especificar consultas manipulando el diagrama. Combinan menús y formularios.
- **Interfaces de lenguaje natural:** Aceptan solicitudes en lenguaje humano (ej. inglés) e intentan interpretarlas, generando una consulta de alto nivel. Si no entienden, inician un diálogo para clarificar.
- **Búsqueda de base de datos basada en palabras clave:** Similar a los motores de búsqueda web, aceptan cadenas de palabras y las asocian con documentos utilizando índices y funciones de clasificación.
- **Entrada y salida de voz:** Uso limitado de voz para consultas y respuestas en aplicaciones con vocabularios restringidos (ej. directorios telefónicos).
- **Interfaces para usuarios paramétricos:** Diseñadas para usuarios que realizan operaciones repetitivas (ej. cajeros bancarios) con comandos abreviados.
- **Interfaces para el DBA:** Contienen comandos privilegiados para tareas de administración, como crear cuentas o cambiar esquemas.

1.6.7 Representación de Relaciones Complejas entre Datos

Una base de datos puede contener una gran variedad de datos interrelacionados. Por ejemplo, el registro de 'Brown' en el archivo STUDENT se relaciona con cuatro registros en GRADE_REPORT. Un **DBMS** debe ser capaz de:

- Representar diversas relaciones complejas entre los datos.
- Definir nuevas relaciones a medida que surgen.
- Recuperar y actualizar datos relacionados de forma fácil y eficiente.

1.6.8 Aplicación de Restricciones de Integridad

La mayoría de las aplicaciones de bases de datos tienen **restricciones de integridad** que los datos deben cumplir. El **DBMS** debe proporcionar capacidades para definir y aplicar estas restricciones. Algunos tipos comunes son:

- **Tipos de datos:** Especificar que un valor debe ser un entero, una cadena de caracteres, etc.
- **Restricciones de unicidad (o clave):** Asegurar que un valor o combinación de valores sea único para cada registro (ej. cada curso debe tener un Número_de_curso único).
- **Restricciones de integridad referencial:** Asegurar que un registro en un archivo esté relacionado con registros en otros archivos (ej. cada registro de sección debe estar relacionado con un registro de curso válido).

Estas restricciones se derivan del significado o semántica de los datos y del **mini-mundo** que representan. Los diseñadores de bases de datos son responsables de identificarlas. Las restricciones pueden ser especificadas al **DBMS** para que las aplique automáticamente, o ser verificadas por programas de actualización. A menudo, las restricciones para grandes aplicaciones se denominan **reglas de negocio**.

Es importante notar que el **DBMS** no puede detectar todos los errores. Por ejemplo, si se ingresa una calificación de 'C' cuando debería ser una 'A', el **DBMS** no lo detecta si 'C' es un valor válido para el tipo de datos Calificación. Sin embargo, rechazaría una calificación de 'Z' si no es un valor válido. Las reglas inherentes a un modelo de datos (por ejemplo, que una relación en el modelo Entidad-Relación involucre al menos dos entidades) son aplicadas implícitamente por el **DBMS**.

1.6.9 Permitiendo la Inferencia y Acciones Usando Reglas y Disparadores

Algunos sistemas de bases de datos permiten definir reglas de deducción para inferir nueva información a partir de los hechos almacenados. Estos se llaman **sistemas de bases de datos deductivas**. Esto permite especificar reglas complejas (ej. cuándo un estudiante está en período de prueba) de forma declarativa, sin tener que escribir código procedimental explícito.

En los sistemas de bases de datos relacionales actuales, pueden asociar **disparadores** (*triggers*) a las tablas. Un **disparador** es una forma de regla que se activa por actualizaciones a la tabla, lo que resulta en la realización de operaciones adicionales en otras tablas, envío de mensajes, etc. **Procedimientos almacenados** son procedimientos más complejos para aplicar reglas, que se invocan cuando se cumplen ciertas condiciones. Los **sistemas de bases de datos activos** ofrecen una funcionalidad aún más potente con reglas activas que inician acciones automáticamente ante eventos y condiciones específicas.

1.6.10 Implicaciones Adicionales del Uso del Enfoque de Bases de Datos

El enfoque de base de datos ofrece otros beneficios significativos para las organizaciones:

- **Potencial para aplicar estándares:** El **DBA** puede definir y aplicar estándares en toda la organización, facilitando la comunicación y cooperación entre departamentos. Esto incluye nombres y formatos de elementos de datos, formatos de visualización y terminología.
- **Reducción del tiempo de desarrollo de aplicaciones:** Aunque el diseño de una base de datos multiusuario grande puede llevar tiempo, una vez operativa, el desarrollo de nuevas aplicaciones se acelera enormemente. Se estima que el tiempo de desarrollo se reduce entre un sexto y un cuarto en comparación con los sistemas de archivos.
- **Flexibilidad:** Los **DBMS** modernos permiten cambios evolutivos en la estructura de la base de datos sin afectar los datos almacenados ni los programas de aplicación existentes, adaptándose a nuevos requisitos.

- **Disponibilidad de información actualizada:** Un **DBMS** hace que la base de datos esté disponible para todos los usuarios, y las actualizaciones realizadas por un usuario son inmediatamente visibles para los demás. Esto es crucial para aplicaciones de procesamiento de transacciones, como sistemas de reservas.
- **Economías de escala:** El enfoque **DBMS** permite consolidar datos y aplicaciones, reduciendo la superposición de actividades y redundancias. Esto posibilita invertir en procesadores, dispositivos de almacenamiento y redes más potentes, reduciendo los costos operativos y de gestión generales.

1.7 Breve Historia de las Aplicaciones de Bases de Datos

Ahora, haremos un repaso rápido por la historia de las aplicaciones que usan **DBMS**, y cómo estas impulsaron el desarrollo de nuevos tipos de sistemas de bases de datos.

1.7.1 Primeras Aplicaciones de Bases de Datos Usando Sistemas Jerárquicos y de Red

Las primeras aplicaciones de bases de datos gestionaban registros en grandes organizaciones (corporaciones, universidades, hospitales). Estas aplicaciones manejaban grandes volúmenes de registros con estructuras similares y numerosas interrelaciones.

Un problema clave de estos primeros sistemas era que mezclaban las relaciones conceptuales con el almacenamiento físico de los registros en disco. Esto limitaba la **abstracción de datos** y la **independencia programa-datos**. Aunque eran eficientes para las consultas originales, era difícil implementar nuevas consultas que requerían una organización de almacenamiento diferente.

Otro inconveniente era que solo ofrecían interfaces de lenguaje de programación, lo que hacía lento y costoso implementar nuevas consultas, pues implicaba escribir y depurar nuevos programas. Estos sistemas, predominantes entre los años 60 y 80, se basaban principalmente en tres paradigmas: **sistemas jerárquicos**, **sistemas basados en el modelo de red** y **sistemas de archivos invertidos**.

1.7.2 Provisión de Abstracción de Datos y Flexibilidad de Aplicaciones con Bases de Datos Relacionales

Las **bases de datos relacionales** surgieron para separar el almacenamiento físico de los datos de su representación conceptual y para proporcionar una base matemática para la representación y consulta de datos. Introdujeron lenguajes de consulta de alto nivel que aceleraron la creación de nuevas consultas. Al principio, los sistemas relacionales (como los **DBMS relacionales (RDBMS)** comerciales de principios de los 80) eran lentos. Sin embargo, con el desarrollo de nuevas técnicas de almacenamiento e indexación y una mejor optimización de consultas, su rendimiento mejoró significativamente.

Eventualmente, las **bases de datos relacionales** se convirtieron en el tipo dominante de sistema de base de datos para las aplicaciones tradicionales, estando presentes en casi todo tipo de computadoras.

1.7.3 Aplicaciones Orientadas a Objetos y la Necesidad de Bases de Datos Más Complejas

La aparición de lenguajes de programación orientados a objetos en los años 80 y la necesidad de almacenar y compartir objetos complejos y estructurados impulsaron el desarrollo de las **bases de datos**

orientadas a objetos (OODBs). Las **OODBs** ofrecían estructuras de datos más generales e incorporaban paradigmas orientados a objetos como tipos de datos abstractos, encapsulación de operaciones, herencia e identidad de objetos.

Sin embargo, su complejidad y la falta de un estándar temprano limitaron su adopción a aplicaciones especializadas, como diseño de ingeniería o sistemas de fabricación. Con el tiempo, muchos conceptos orientados a objetos se incorporaron a las nuevas versiones de los **DBMS** relacionales, dando lugar a los **sistemas de gestión de bases de datos objeto-relacionales (ORDBMSs).**

1.7.4 Intercambio de Datos en la Web para el Comercio Electrónico Usando XML

La *World Wide Web*, con su red de computadoras interconectadas, vio el surgimiento del comercio electrónico (*e-commerce*) en los años 90. Gran parte de la información crucial en las páginas web de comercio electrónico se extrae dinámicamente de los **DBMS** (como información de vuelos o precios de productos).

El **Lenguaje de Marcado Extensible (XML)** se convirtió en un estándar para intercambiar datos entre diversos tipos de bases de datos y páginas web. **XML** combina conceptos de modelos de documentos con conceptos de modelado de bases de datos. Veremos una descripción general de **XML** en el *Capítulo 13*.

1.7.5 Extensión de las Capacidades de las Bases de Datos para Nuevas Aplicaciones

El éxito de los sistemas de bases de datos en aplicaciones tradicionales incentivó a desarrolladores de otros tipos de aplicaciones a usarlos, incluso si estas tradicionalmente usaban su propio software especializado. Los sistemas de bases de datos ahora ofrecen extensiones para satisfacer las necesidades específicas de estas nuevas aplicaciones. Algunos ejemplos son:

- **Aplicaciones científicas:** Almacenan grandes volúmenes de datos de experimentos (física, genoma humano).
- **Almacenamiento y recuperación de imágenes:** Fotografías, imágenes satelitales, radiografías.
- **Almacenamiento y recuperación de videos.**
- **Aplicaciones de minería de datos:** Analizan grandes volúmenes de datos para buscar patrones o relaciones, o identificar anomalías (ej. detección de fraude con tarjetas de crédito).
- **Aplicaciones espaciales:** Almacenan y analizan ubicaciones geográficas (mapas, sistemas de navegación).
- **Aplicaciones de series temporales:** Almacenan datos económicos en puntos regulares de tiempo (ventas diarias).

Los sistemas relacionales básicos a menudo no eran adecuados para estas aplicaciones debido a la necesidad de estructuras de datos más complejas, nuevos tipos de datos, nuevas operaciones y construcciones de lenguaje de consulta, y nuevas estructuras de almacenamiento e indexación. Esto llevó a los desarrolladores de **DBMS** a añadir funcionalidades, ya sea de propósito general (como incorporar conceptos orientados a objetos) o especializadas (módulos opcionales para aplicaciones específicas).

1.7.6 Surgimiento de Sistemas de Almacenamiento de Big Data y Bases de Datos NOSQL

En la primera década del siglo XXI, la proliferación de aplicaciones y plataformas como las redes sociales, el comercio electrónico masivo, los índices de búsqueda web y el almacenamiento en la nube, provocó un aumento masivo en la cantidad de datos almacenados. Esto requirió nuevos tipos de sistemas de bases de datos que ofrecieran búsqueda y recuperación rápidas, así como almacenamiento confiable y seguro de datos no tradicionales, como publicaciones de redes sociales.

Algunos de los requisitos de estos nuevos sistemas no eran compatibles con los **DBMS** relacionales SQL (el estándar para bases de datos relacionales). El término **NOSQL** se interpreta generalmente como "Not Only SQL" (No Solo SQL), lo que significa que en sistemas que gestionan grandes volúmenes de datos, una parte puede almacenarse con sistemas SQL y otra con sistemas **NOSQL**, según las necesidades de la aplicación. Esto lo veremos en detalle en el *Capítulo 24*.

1.8 Cuándo No Usar un DBMS

A pesar de las múltiples ventajas de usar un **DBMS**, hay situaciones en las que su uso puede implicar costos generales innecesarios que no se incurrirían con el procesamiento de archivos tradicional. Estos costos se deben a:

- Alta inversión inicial en hardware, software y capacitación.
- La generalidad que un **DBMS** proporciona para definir y procesar datos.
- Los costos generales de proporcionar funciones de seguridad, control de concurrencia, recuperación e integridad.

Por lo tanto, puede ser más conveniente desarrollar aplicaciones de bases de datos personalizadas bajo las siguientes circunstancias:

- Aplicaciones de bases de datos simples y bien definidas que no se espera que cambien.
- Requisitos estrictos de tiempo real para algunos programas de aplicación que no pueden cumplirse debido a la sobrecarga del **DBMS**.
- Sistemas empotrados con capacidad de almacenamiento limitada, donde un **DBMS** de propósito general no encajaría.
- Ausencia de acceso multiusuario a los datos.

Algunas industrias y aplicaciones han optado por no usar **DBMS** de propósito general, desarrollando su propio software de gestión de archivos y datos, optimizado para sus necesidades específicas. Ejemplos incluyen herramientas de diseño asistido por computadora (CAD), sistemas de comunicación y conmutación, o implementaciones de sistemas de información geográfica (GIS).

1.9 Resumen

En este primer capítulo, hemos establecido las bases de nuestra comprensión de los sistemas de bases de datos. Hemos definido una **base de datos** como una colección de datos relacionados que representan un aspecto del mundo real y se utiliza con propósitos específicos.

Aprendimos que un **DBMS** es un paquete de software generalizado que permite implementar y mantener una base de datos computarizada, y que la base de datos y el software juntos forman un **sistema de base de datos**.

Discutimos las características que distinguen el enfoque de bases de datos del procesamiento de archivos tradicional, y categorizamos a los **actores en escena** (como el **DBA**, los diseñadores de bases de datos y los usuarios finales) y a los **trabajadores tras bambalinas** (como los diseñadores de sistemas **DBMS**).

Revisamos una lista de capacidades esenciales que un **DBMS** debe proporcionar, tales como:

- Control de la redundancia.
- Restricción del acceso no autorizado.
- Provisión de almacenamiento persistente para objetos de programa.
- Estructuras de almacenamiento eficientes y técnicas de búsqueda.
- Copias de seguridad y recuperación.
- Múltiples interfaces de usuario.
- Representación de relaciones complejas.
- Aplicación de restricciones de integridad.
- Permitiendo la inferencia y acciones usando reglas y disparadores.

Además, hemos visto otras implicaciones ventajosas del uso de **DBMS**, como la capacidad de aplicar estándares, la reducción del tiempo de desarrollo, la flexibilidad, la disponibilidad de información actualizada y las economías de escala.

Finalmente, les proporcioné una breve perspectiva histórica de la evolución de las aplicaciones de bases de datos, desde los primeros **sistemas jerárquicos y de red** hasta el surgimiento de las **bases de datos relacionales**, los **sistemas orientados a objetos**, **XML**, y la reciente aparición de los sistemas de **big data** y **NOSQL**. También analizamos las situaciones en las que el uso de un **DBMS** podría no ser la opción más ventajosa debido a sus costos generales.

Espero que esta introducción les haya dado una visión clara de la importancia y complejidad de los sistemas de bases de datos.